Laporan Studi Kasus: BCM Developer Intern

Aplikasi Web Daftar Tugas Harian

Kandidat	Maulana Asykari Muhammad
Posisi	BCM Developer Intern, MagangHub CIMB Niaga
Repository	https://github.com/WeissCurry/cimb-maulana
Tanggal	6 November 2026

---

0. Ringkasan

Studi kasus ini mencakup dua soal:

1. Membangun aplikasi web pencatatan daftar tugas harian dengan Python dan MySQL, lengkap dengan dokumentasi instalasi, tabel rute, skema data, dan cara menjalankan test.
2. Menjelaskan komponen utama yang divalidasi pada respons API menggunakan Postman beserta contoh kodenya.

Saya membangun aplikasi web berbasis Flask 3 dan MySQL 8.4 yang menyatukan empat tampilan (Papan Kanban 5 tahap, Daftar baris, Kalender tenggat bulanan, dan Bagan Gantt) dalam satu halaman dengan parameter `?view=`. Formulir input menggunakan dialog modal native `<dialog>`. Tampilan dibangun dengan Tailwind CSS v4 tanpa Node.js, ditambah animasi GSAP 3 dengan pelindung aksesibilitas `prefers-reduced-motion`.

Data hasil pengujian dari berkas bukti:

- 95 test pytest lulus dalam 0.61 detik, mencakup model, validasi payload, logika filter, pengurutan, batas keterlambatan, dan penanganan error.
- 81 asersi Newman Postman lulus tanpa kegagalan (0 failed) pada 16 request API dengan rata-rata waktu respons 2ms dan durasi total 233ms.
- 84 kasus uji ekstrem (injeksi, batas data, konkurensi 100 request paralel) lulus semua, setelah 2 bug yang ditemukannya diperbaiki.
- 24 kombinasi ukuran layar dan tampilan teruji bebas pergeseran samping (`sx: 0`) dari lebar 280px hingga 1440px tanpa error JavaScript.
- Tampilan Kalender desktop pas pada satu layar (`scrollHeight == innerHeight` pada resolusi 1440x900, 1280x720, 1920x1080, 1024x768, dan 884x1104).

---

1. Setup dan Asumsi

Saya menggunakan Python 3.12, Flask 3.1, dan MySQL 8.4 lokal di macOS. Server berjalan pada port 5001 dengan perintah `flask --app app run --debug --port 5001`. Port 5000 dihindari karena dipakai oleh AirPlay Receiver. Untuk pengujian otomatis pada `test_app.py`, saya memakai database SQLite in-memory (`sqlite:///:memory:`) agar cepat dan terisolasi.

Tabel asumsi teknis:

#	Asumsi	Alasan dan Implikasi
A1	Tanpa autentikasi (no auth).	Batasan penugasan; data dapat diakses langsung untuk evaluasi lokal.
A2	Tanpa proteksi CSRF (no CSRF).	Instruksi penugasan untuk menyederhanakan pengujian form dan request API.
A3	Zona waktu Asia/Jakarta (WIB).	Operasional CIMB Niaga berpusat di Indonesia; evaluasi tanggal hari ini mengacu pada WIB.
A4	Status tugas memakai 5 tahap.	Menggantikan sistem biner todo/done agar mencerminkan alur kerja operasional bank.
A5	MySQL 8.4 untuk server dan SQLite untuk test.	Memastikan relasi data riil di lokal sekaligus mempercepat eksekusi pytest.
A6	Styling mandiri dengan Tailwind CSS v4.	Menggunakan binary di `bin/tailwindcss`; file `static/tailwind.css` sudah prebuilt.

---

2. Pemahaman Kebutuhan

Dalam divisi Business Continuity Management (BCM) dan operasional bank, pencatatan tugas harian berfungsi sebagai pengendali alur kerja dan mitigasi risiko operasional. Tugas seperti pengecekan kelengkapan berkas KYC, rekonsiliasi transaksi, pembaruan dokumen SOP kontinjensi, atau pengujian disaster recovery memerlukan pembagian tahap pengerjaan yang jelas, batas waktu tegas, dan jejak audit yang akuntabel.

Saya merancang empat tampilan untuk memenuhi kebutuhan tersebut:

1. **Papan Kanban 5 Tahap:** Memberikan kejelasan posisi pengerjaan tugas (Belum Mulai, Dikerjakan, Siap Dicek, Review, Selesai). Pemisahan tahap Siap Dicek dan Review membedakan tugas yang selesai dikerjakan staf dengan tugas yang sedang diperiksa supervisor.
2. **Riwayat Tahapan (`task_events`):** Menyimpan jejak audit perpindahan status, mencatat durasi kerja pada tiap tahap, dan menghitung total waktu penyelesaian tugas.
3. **Kalender Bulanan:** Memetakan tanggal jatuh tempo tugas operasional perbankan agar beban kerja bulanan terpantau jelas.
4. **Bagan Gantt:** Menampilkan rentang hari pengerjaan dari tanggal buat hingga tenggat waktu untuk mendeteksi potensi tugas terlambat sejak dini.

---

3. Desain Solusi dan Trade-off

Saya memilih stack Python 3.12, Flask 3, Flask-SQLAlchemy, PyMySQL, dan MySQL 8.4. Untuk manipulasi waktu dan kalender, saya memakai modul bawaan Python `zoneinfo` dan `calendar` tanpa dependensi luar.

Model data menggunakan dua tabel relasional:

- Tabel `tasks`: Menyimpan data tugas (`id`, `title`, `description`, `due_date`, `priority`, `status`, `created_at`, `updated_at`).

- Tabel `task_events`: Mencatat riwayat status (`id`, `task_id`, `from_status`, `to_status`, `created_at`) dengan foreign key berkaskade (`ON DELETE CASCADE`) dan indeks `(task_id, created_at)`.

Aturan status: Pembuatan tugas mencatat satu event awal (`from_status: null`). Setiap perubahan status mencatat satu event baru. Pemindahan ke status yang sama tidak dicatat. Format waktu JSON selalu UTC ISO 8601 berakhiran `Z` (contoh `2026-11-06T10:00:00Z`).

Keputusan antarmuka:

- Server-Rendered Jinja: Seluruh HTML dirender di server dengan fallback `/?view=<name>`. JavaScript menambahkan navigasi tab tanpa reload (`history.replaceState`), kontrol dialog, dan drag and drop.
- Tailwind CSS v4 Standalone: Kompilasi berjalan via binary `bin/tailwindcss` tanpa Node.js. File `static/tailwind.css` sudah prebuilt di repositori.
- Aksesibilitas Warna CIMB: Warna merah `#EC2428` memiliki rasio kontras 4.33:1 terhadap teks putih, di bawah standar WCAG AA (minimal 4.5:1). Saya menyesuaikan tombol primer menjadi `#D61E22` (kontras 5.16:1) agar lolos uji WCAG AA.
- Komponen Terpusat: Seluruh tombol dibuat lewat makro Jinja di `templates/components/ui.html`.
- Animasi GSAP 3: Transisi tombol dan dialog memakai GSAP 3 via CDN, dengan pelindung `prefers-reduced-motion: reduce`.

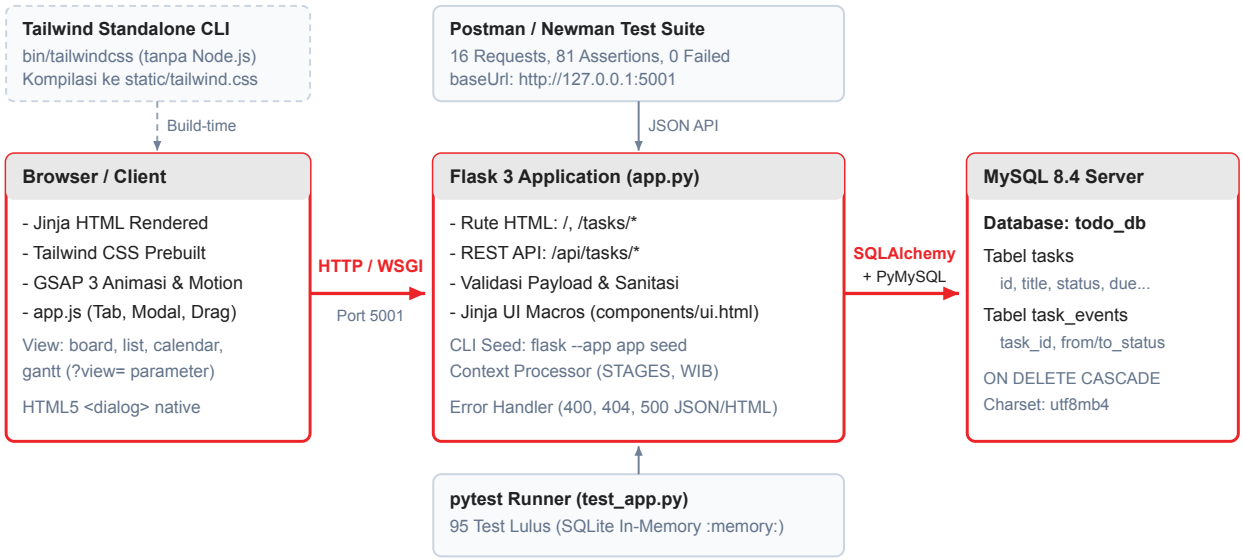
Tabel perbandingan keputusan:

Aspek	Pilihan Terpilih	Alternatif yang Ditolak	Alasan
Halaman	Satu halaman (`app.html`) dengan sub-view	Halaman terpisah per rute	Menghilangkan reload penuh antar tampilan sehingga alur terpantau dalam satu konteks.
Styling	Binary standalone Tailwind v4	Node.js + npm pipeline	Menghindari instalasi node_modules; proyek dapat dijalankan murni dengan Python.
Dialog Form	Native HTML5 `<dialog>`	Modal kustom div overlay	Memakai aksesibilitas bawaan browser (focus trap, tombol Escape, backdrop native).
Waktu	UTC DATETIME di database, format ISO 8601 Z di JSON	Waktu lokal tanpa zona waktu	Menghindari perbedaan waktu lokal antar klien dan mematuhi standar ISO.

---

4. Arsitektur Sistem dan Tech Stack

Berikut adalah arsitektur sistem dan alur data aplikasi web daftar tugas harian:



Gambar 1. Diagram arsitektur sistem dan alur data aplikasi.

Tabel tech stack:

Layer	Teknologi	Versi	Alasan
Bahasa Pemrograman	Python	3.12	Bahasa utama backend dengan dukungan pengetikan dan zoneinfo bawaan.
Web Framework	Flask	3.0.3	Framework WSGI minimalis, cepat, dan fleksibel untuk rendering rute HTML dan API REST.
ORM Database	Flask-SQLAlchemy	3.1.1	Abstraksi relasional tabel tasks dan task_events dengan manajemen sesi otomatis.
Database Driver	PyMySQL	1.1.1	Driver MySQL berbasis Python murni tanpa ketergantungan library C lokal.
Basis Data	MySQL	8.4 LTS	RDBMS relasional ACID standar produksi perbankan dengan integritas foreign key kaskade.
Template Engine	Jinja2	3.1.4	Server-side rendering aman dengan escape HTML otomatis dan modularitas makro UI terpusat.
Styling CSS	Tailwind CSS	v4.0 Standalone	Utilitas CSS modern terkompilasi melalui binary tanpa ketergantungan ekosistem Node.js.
Animasi Antarmuka	GSAP	3.12.5	Library animasi web dengan pelindung aksesibilitas prefers-reduced-motion.
Tipografi	Plus Jakarta Sans	Google Fonts	Font geometri modern standar korporat perbankan Indonesia dengan keterbacaan tinggi.
Pengujian Backend	pytest	8.3.3	Framework otomatisasi 95 test unit dan rute dengan isolasi basis data SQLite in-memory.
Pengujian API	Postman / Newman	Postman v2.1 / Newman 6	Otomasi 81 asersi validasi kontrak skema JSON, batas waktu, dan kode status HTTP.
Pengujian UI	Playwright	1.48.0	Headless browser Chromium untuk verifikasi 24 kombinasi resolusi tanpa pergeseran samping.

5. Implementasi

Aplikasi berjalan pada file `app.html` yang menampung header, tab navigasi, filter prioritas, tombol tambah tugas, dan modal dialog:

- Tampilan Papan (`views/board.html`): 5 kolom alur kerja dengan kartu tugas, chip prioritas, due date, tombol geser status, dan drag and drop.
- Tampilan Daftar (`views/list.html`): Format baris data pada desktop (>= 600px) dan kartu bertumpuk pada ponsel (< 600px).
- Tampilan Kalender (`views/calendar.html`): Kisi bulanan 7 kolom Senin-Minggu pada desktop dengan batas 3 tugas per tanggal, serta daftar agenda pada ponsel (< 600px).
- Bagan Gantt (`views/gantt.html`): Baris visual rentang hari pengerjaan dengan batas maksimal 45 hari.

Antarmuka menyediakan *loading skeletons* saat pergantian tampilan untuk menjaga stabilitas visual sebelum konten selesai dimuat. Untuk mengisi data awal, perintah CLI `flask --app app seed --reset` memuat 14 contoh tugas operasional perbankan lengkap dengan riwayat tahapan.

Tabel rute HTML:

Method	Path	Deskripsi	Status
GET	`/`	Halaman utama. Query: `view` , `priority` , `month` , `task` .	200
POST	`/tasks`	Tambah tugas dari modal. Valid: redirect. Invalid: 400.	302 / 400
GET	`/tasks/<id>`	Detail tugas. Redirect 302 ke `/?task=<id>` .	302
GET	`/tasks/<id>/edit`	Form ubah. Redirect 302 ke `/?task=<id>` .	302
POST	`/tasks/<id>/edit`	Simpan ubah tugas dan status.	302 / 400
POST	`/tasks/<id>/move`	Pindah status tahapan.	302
POST	`/tasks/<id>/delete`	Hapus tugas dan riwayat transisi.	302
GET	`/calendar`	Redirect 302 ke `/?view=calendar` .	302

Tabel endpoint REST API (`/api/tasks`):

Method	Path	Deskripsi	Sukses	Error
GET	`/api/tasks`	Daftar tugas dengan filter opsional `status` dan `priority` .	200 `{"data":[...], "count":n}`	400
POST	`/api/tasks`	Buat tugas baru. Mengembalikan header `Location` .	201 `{"data":task}`	400
GET	`/api/tasks/<id>`	Data satu tugas berdasarkan ID.	200 `{"data":task}`	404

Method	Path	Deskripsi	Sukses	Error
PUT	`/api/tasks/<id>`	Update atribut tugas parsial atau penuh.	200 `{"data":task}`	400 / 404
POST	`/api/tasks/<id>/move`	Pindahkan tahapan tugas dengan payload `{"status": key}`.	200 `{"data":task}`	400 / 404
GET	`/api/tasks/<id>/history`	Riwayat transisi status tugas terurut dari paling awal.	200 `{"data":[...]}`	404
DELETE	`/api/tasks/<id>`	Hapus data tugas beserta riwayat transisinya.	200 `{"message":"Tugas dihapus"}`	404

Perilaku responsif:

- Desktop (>= 1280px): 5 kolom Kanban sejajar, kalender fit satu layar tanpa scroll window utama, dan Gantt dengan sticky header.
- Layar lipat terbuka (673px dan 884px) serta tablet (1024px): Papan Kanban digeser horizontal via `scroll-snap-type: x mandatory` pada lebar kolom ~300px tanpa menggeser halaman utama.
- Layar ponsel (280px dan 390px): Satu kolom Kanban tampil per layar (~88% lebar) dengan tombol anchor tahapan di atas. Kalender beralih ke agenda. Modal dialog menjadi bottom sheet setinggi `100dvh`.

---

6. Bukti Pengujian

6.1 Bukti Pytest (95 Passed)

Berkas bukti: evidence/pytest.txt

```
$ .venv/bin/python -m pytest -v  (2026-11-06 19:28 WIB)
collected 95 items
...
============================== 95 passed in 0.61s ==============================
```

Seluruh pengujian unit dan rute backend lulus tanpa kesalahan.

6.2 Bukti Newman Postman (81 Assertions, 0 Failed)

Berkas bukti: evidence/postman-newman.txt

```
$ newman run postman_collection.json  (2026-11-06 19:02 WIB)
iterations: 1, requests: 16, test-scripts: 16, assertions: 81 (executed: 81, failed: 0)
total run duration: 233ms, average response time: 2ms [min: 1ms, max: 14ms]
```

Sebanyak 16 request API berhasil divalidasi dengan 81 asersi tanpa kegagalan.

6.3 Bukti Interaksi UI dan Viewport Playwright (Zero Sideways Scroll)

Berkas bukti: evidence/ui-click-through.txt

```
Playwright click-through check, Chromium headless (2026-11-06 19:00 WIB)
tab list, calendar, gantt, board -> all visible True
create dialog modal=True, page scrolled: False, escape closes: True
viewports 280, 390, 673, 884, 1024, 1440 across 4 views -> all sx: 0, small: []
js errors: []
```

Pengujian memastikan tab berpindah mulus, dialog modal mengunci scroll halaman, 24 kombinasi viewport bebas scroll horizontal (`sx: 0`), dan ukuran tombol memenuhi minimal 44x44px.

6.4 Bukti Fit Kalender Desktop (ScrollHeight == InnerHeight)

Berkas bukti: evidence/calendar-fit.txt

```
Calendar view fit check: scrollHeight (sh) must equal innerHeight (ih) on desktop
1440 900 {'sh': 900, 'ih': 900, 'sx': False}
1280 720 {'sh': 720, 'ih': 720, 'sx': False}
1920 1080 {'sh': 1080, 'ih': 1080, 'sx': False}
1024 768 {'sh': 768, 'ih': 768, 'sx': False}
884 1104 {'sh': 1104, 'ih': 1104, 'sx': False}
errors []
```

Pada resolusi desktop dan tablet, tinggi total halaman sama dengan tinggi layar peramban (`sh == ih`), sehingga kalender pas satu layar tanpa scrollbar jendela.

6.x Pengujian Ekstrem

Saya menjalankan pengujian kasus batas dan beban ekstrem pada backend dan antarmuka pengguna:

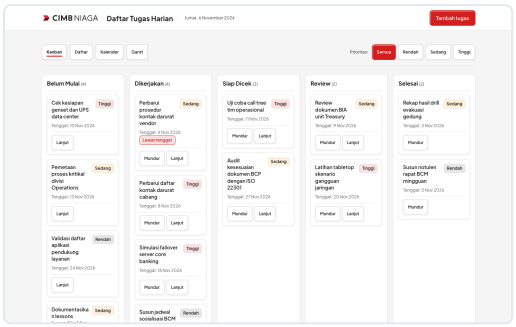
- Pengujian ekstrem API dan keamanan backend (evidence/extreme-tests.txt): Sebanyak 84 skenario dijalankan dengan hasil 84 lulus dan 0 gagal (84 pass, 0 fail). Pengujian mencakup payload XSS dan SQL injection, batas panjang dan karakter Unicode (200 karakter CJK, emoji, teks RTL), tanggal tidak valid termasuk 2026-02-29 serta tahun 0001 dan 9999, body request rusak dan body 1 MB, ID bernilai sangat besar, HTTP method yang salah, percobaan open-redirect, konkurensi 100 create dan 100 move paralel, serta performa 400+ tugas di mana setiap tampilan merespons di bawah 1.5 detik. Dua bug berhasil ditemukan dan diperbaiki: bulan kalender 9999-12 dan 0001-01 yang memicu error 500 karena overflow tanggal, serta karakter garis miring terbalik (backslash) pada parameter redirect yang sempat diterima.
- Pengujian ekstrem antarmuka pengguna (evidence/ui-extreme.txt): Judul tugas berisi payload XSS dipastikan tidak pernah tereksekusi (0 alert, 0 JavaScript error) pada 4 tampilan dan 4 ukuran layar (280px, 390px, 884px, 1440px). Judul panjang 200 karakter tanpa spasi yang awalnya meluap pada tabel daftar dan panel kalender telah diperbaiki sehingga menghasilkan 0 elemen meluap (0 escaping elements).

---

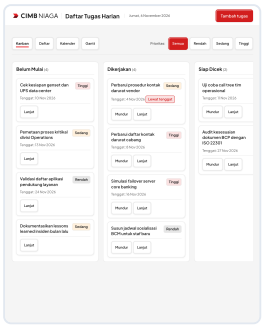
7. Bukti Visual

Tangkapan layar hasil uji Playwright Chromium headless di direktori `assessment/screenshots`:

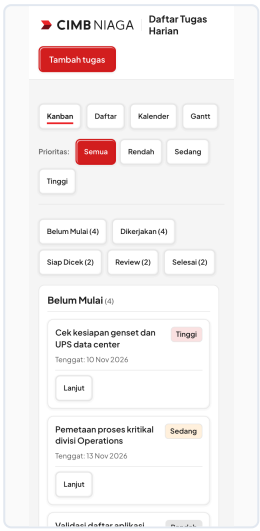
7.1 Tampilan Papan Kanban (Board View)



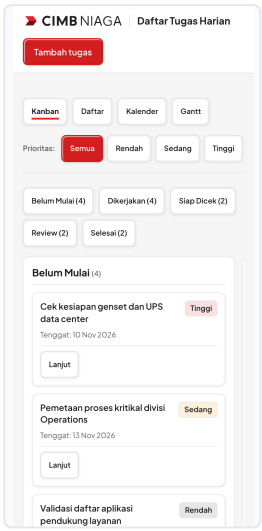
(a) Desktop 1440px



(b) Fold terbuka 884px



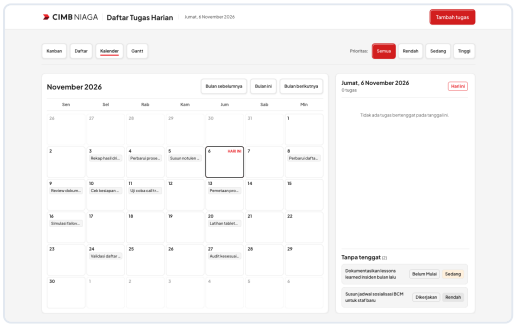
(c) Fold tertutup 344px



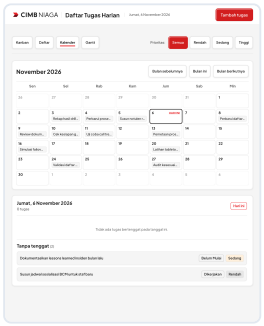
(d) Ponsel 390px

Gambar 2. Tampilan Papan Kanban: (a) desktop, (b) fold terbuka, (c) fold tertutup, (d) ponsel.

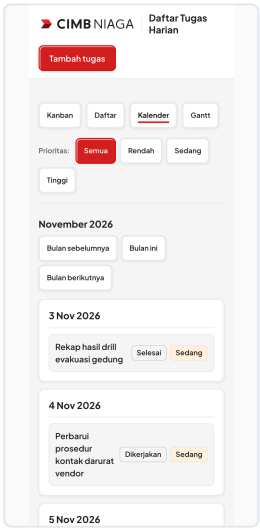
7.2 Tampilan Kalender Bulanan (Calendar View)



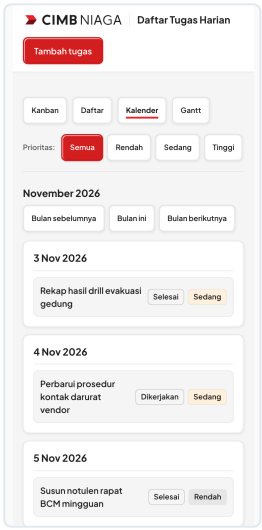
(a) Desktop 1440px



(b) Fold terbuka 884px



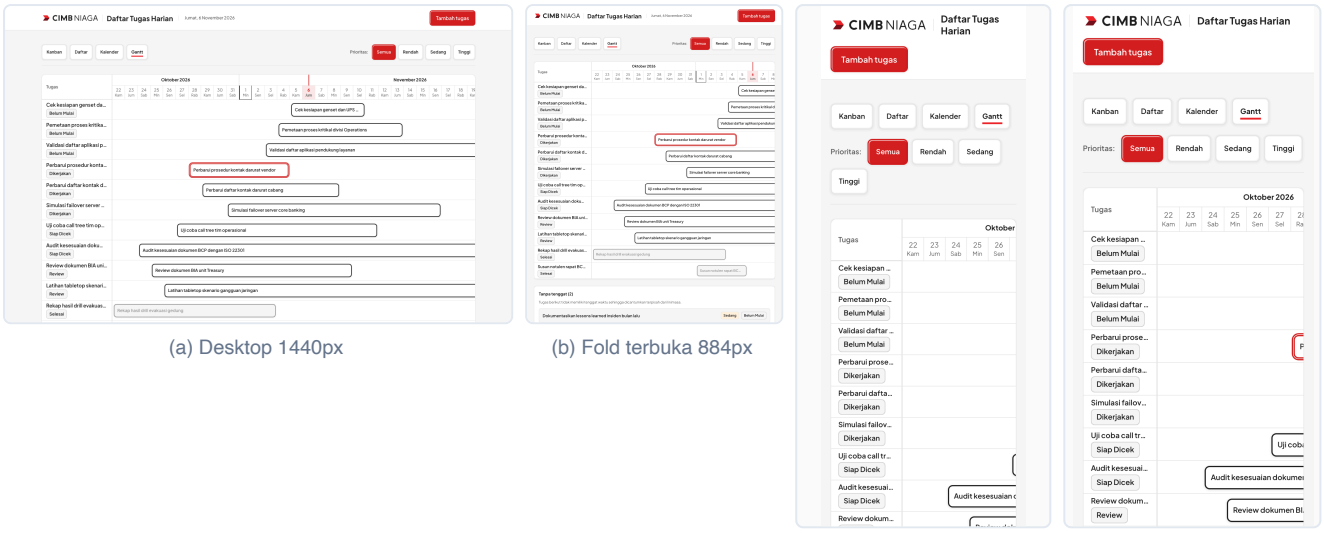
(c) Fold tertutup 344px



(d) Ponsel 390px

Gambar 3. Tampilan Kalender Bulanan: (a) desktop, (b) fold terbuka, (c) fold tertutup, (d) ponsel.

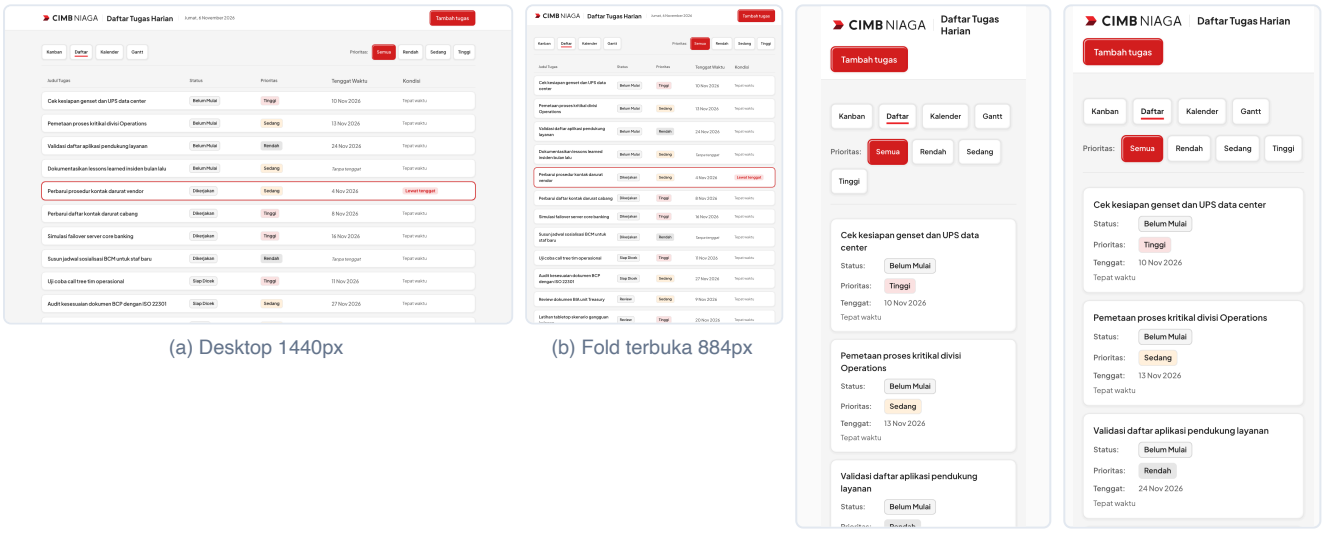
7.3 Tampilan Bagan Gantt (Gantt View)



(a) Desktop 1440px (b) Fold terbuka 884px (c) Fold tertutup 344px (d) Ponsel 390px

Gambar 4. Tampilan Bagan Gantt: (a) desktop, (b) fold terbuka, (c) fold tertutup, (d) ponsel.

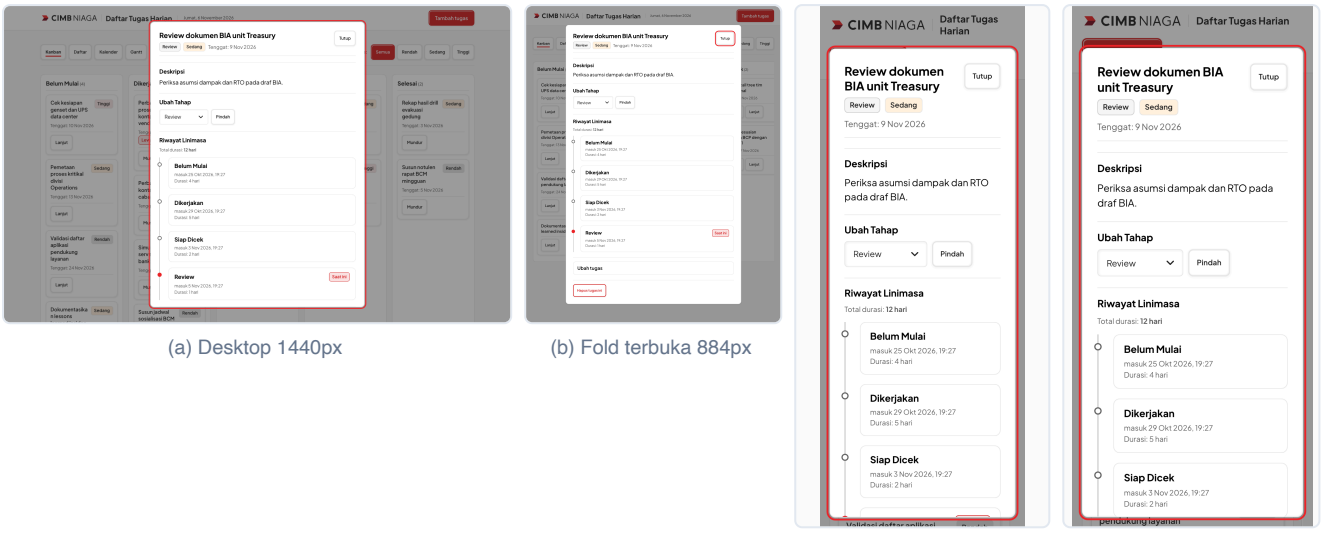
7.4 Tampilan Tabel Daftar (List View)



(a) Desktop 1440px (b) Fold terbuka 884px (c) Fold tertutup 344px (d) Ponsel 390px

Gambar 5. Tampilan Tabel Daftar: (a) desktop, (b) fold terbuka, (c) fold tertutup, (d) ponsel.

7.5 Modal Dialog Rincian Tugas dan Linimasa



(a) Desktop 1440px (b) Fold terbuka 884px (c) Fold tertutup 344px (d) Ponsel 390px

Gambar 6. Modal Dialog Rincian Tugas: (a) desktop, (b) fold terbuka, (c) fold tertutup, (d) ponsel.

---

8. Jawaban Soal QA (Postman API Testing)

Saya memvalidasi sembilan komponen utama pada setiap respons API:

1. **Status Code:** Memeriksa kode status HTTP (200, 201, 400, 404).
2. **Response Time:** Memastikan waktu pemrosesan di bawah 500ms.
3. **Response Headers:** Memverifikasi `Content-Type: application/json` dan header `Location`.
4. **Body Structure / JSON Schema:** Menguji skema JSON dengan `pm.response.to.have.jsonSchema`.
5. **Data Types and Values:** Mengecek tipe data dan batasan nilai enum (5 tahap alur kerja, prioritas, akhiran waktu Z).
6. **Business Rules:** Memverifikasi logika bisnis, termasuk flag `is_overdue` aktif jika status belum selesai dan tenggat lampau.
7. **Error Handling and Messages:** Memastikan pesan error terstruktur pada objek `error.message` dan `error.fields`.
8. **Authentication and Cookies:** Memeriksa akses publik tanpa header otorisasi dan ketiadaan cookie sesi.
9. **Consistency with Request:** Memastikan data yang disimpan dan dikembalikan server cocok dengan body request.

Penjelasan lengkap dan contoh kode pengujian Postman tersedia di berkas JAWABAN_QA.md.

---

9. Penggunaan AI dan Verifikasinya

Saya mengembangkan proyek ini dengan bantuan AI secara transparan:

- Claude Code sebagai koordinator arsitektur melalui harness Herdr untuk mengelola spesifikasi dan alur pengerjaan.

- Agen Antigravity dan OpenCode sebagai eksekutor kode backend Flask, komponen Jinja dan Tailwind v4, serta pengujian otomatis.

Setiap kode diverifikasi melalui tiga tahap: 95 test pytest, 81 asersi Newman Postman, dan skrip Playwright headless browser.

Review menemukan dan memperbaiki enam bug sebelum rilis:

1. **Form edit menimpa status done:** Form awal tidak menyertakan status sehingga tugas done kembali ke todo. Saya menambahkan pilihan status dan validasi parsial.
2. **Variabel responseBody bentrok di Postman:** Deklarasi const responseBody menimpa variabel global Postman sehingga 12 asersi gagal. Saya mengganti namanya menjadi createdBody.
3. **Scope loop Jinja pada navigasi tahapan:** Variabel indeks tidak terbarui di luar loop Jinja sehingga tombol Mundur hilang. Saya memperbaikinya dengan namespace(idx=-1).
4. **Overflow horizontal pada tablet:** Kontainer papan memicu scrollbar horizontal pada lebar 600px sampai 1023px. Saya mengisolasinya dengan scroll-snap-type dan overflow-x: auto.
5. **Dialog server terbuka non-modal:** Saat validasi form gagal, server me-render ulang dialog tanpa memanggil showModal(). Saya menambahkan pemanggilan showModal() dan kelas .is-locked.
6. **Kalender melebihi tinggi layar desktop:** Tabel kalender awal melebihi tinggi layar. Saya memperbaikinya dengan flexbox adaptif agar scrollHeight sama dengan innerHeight.

---

10. Langkah Pengembangan Berikutnya

Jika waktu pengerjaan lebih panjang, saya akan melakukan uji responsivitas pada perangkat fisik (termasuk ponsel lipat dengan engsel) dan audit Lighthouse untuk performa, aksesibilitas, best practices, dan SEO, lalu melampirkan hasilnya sebagai bukti tambahan.

Rencana peningkatan lanjutan untuk kebutuhan produksi perbankan:

1. **Autentikasi dan Kontrol Akses (RBAC):** Integrasi Single Sign-On (SSO) LDAP perbankan dengan pemisahan peran antara Staf, Reviewer, Koordinator BCM, dan Auditor.
2. **Keamanan:** Penambahan token CSRF pada form HTML dan pembatasan laju request (rate limiting) pada API.
3. **Notifikasi:** Pengiriman email atau webhook otomatis saat tugas mendekati batas tenggat waktu.
4. **Migrasi Database:** Integrasi Alembic (Flask-Migrate) untuk mengelola riwayat perubahan skema tanpa drop tabel.
5. **Otomasi CI/CD:** Pipeline GitHub Actions untuk menjalankan pytest, Newman Postman, linter, dan uji visual Playwright pada setiap pull request.

---

11. Sumber dan Atribusi

- Logo, favicon, dan palet warna CIMB Niaga diambil dari `cimbniaga.co.id` untuk studi kasus penugasan MagangHub ini saja, bukan produk resmi PT Bank CIMB Niaga Tbk.
- Tailwind CSS v4 standalone executable binary dari Tailwind Labs tanpa dependensi Node.js.
- GSAP 3 versi 3.12.5 via CDN jsDelivr.
- Font Plus Jakarta Sans dari Google Fonts, dirancang oleh Tokotype (Gumpita Rahayu).